

C++ Template 延伸自主學習報告：STL 與基礎資料結構入門

1. 自主學習主題與目標

在這學期學完 C++ 的基礎語法、物件導向（OOP）與 Chapter 16 Templates 的觀念後，我發現寫程式時常常需要重複造輪子。例如為了存不同型別的資料，就要自己刻好幾次 Dynamic Array 或是 Linked List。

本次兩週自主學習的核心目標，就是**延伸 Template 的 Function 與 Class 觀念，進一步跨入 C++ 標準模板函式庫（STL）的世界**。透過實作「學生成績管理系統」，我希望能達到以下目標：

- 理解 STL 容器（Containers）與演算法

（Algorithms）底層是如何透過 Template 實現泛型編程的。

- 提早熟悉 `vector`、`stack`、`queue`、`map` 等常用容器的操作與其對應的基礎資料結構。
- 學習如何在符合 `g++ -std=c++17` 的環境下進行專案的架構設計與編譯管理，為大二即將面臨的重頭戲「資料結構」與「演算法」課程打下扎實的先備基礎。

2. Template 與 STL 的關係

在學習 STL 之前，我們學過可以用 `template <class T>` 來讓函式或類別套用於不同的型別。而 STL（Standard Template Library）其實就是 C++ 官方幫我們寫好的、極致的 Template 實際應用。

舉例來說，當我們宣告 `std::vector<int>` 或

`std::vector<string>` 時，這群動態陣列的底層行為（如 `push_back()`、記憶體配置）邏輯完全一模一樣，只有裡面裝的資料型別不同。STL 利用 Class Template，讓我們在開發時不用去管繁瑣的記憶體配置與型別轉換，隔離了型別差異，實現了程式碼的高度重用性（Code Reusability）。

容器名稱	底層資料結構	常見核心操作	時間複雜度
<code>std::vector</code>	動態連續陣列	<code>push_back()</code> , <code>[]</code>	尾端插入 $O(1)$ 隨機存取 $O(1)$
<code>std::stack</code>	封裝的雙端佇列	<code>push()</code> , <code>pop()</code> , <code>top()</code>	插入/刪除 $O(1)$
<code>std::queue</code>	封裝的雙端	<code>push()</code> ,	插入/刪除

	佇列	<code>pop()</code> , <code>front()</code>	$O(1)$
--	----	--	--------

4. 第 2 週小專題設計說明

第二週的小專題是實作一個「學生成績管理系統」。

4.1 資料結構的抉擇：`vector` vs `map`

作業指引提到可以使用 `std::vector<Student>` 或 `std::map<string, Student>`。在設計架構時，我陷入了卡關與思考：

- 如果選用 `std::map`：用學號當 Key 可以讓「功能 4：查詢學生」變得非常快（只要 $O(\log n)$ ），而且天生能防止學號重複的問題。但是，當走到「功能 3：依成績排序」

時，因為 `map` 預設是依照 Key（學號）排序，我必須大費周章地把資料全部倒進一個 `vector` 裡，才能呼叫 `std::sort()`。

- 如果選用 `std::vector`：雖然在「功能 1：新增學生」時需要寫一個迴圈去檢查學號有沒有重複（時間複雜度 $O(n)$ ），但是「功能 3：成績排序」與「功能 5：統計」可以直接配合 STL algorithms 進行操作。

最終決定：考慮到這是一個「成績管理系統」，對成績進行排序與統計的頻率與重要性極高，因此我最終選用 `std::vector<Student>` 作為主要的儲存容器，並自行用迴圈補足學號重複檢查的邏輯。

5. 主要程式架構與重要程式片段

本系統在 `main()` 函式中採用一個 `while` 迴圈與 `switch-case` 搭建出 CLI 選單介面。以下是我認為專案中最關鍵的兩個程式片段：

5.1 成績排序與 Lambda 運算式 (功能 3)

在實作依成績由高到低排序時，我使用了 `<algorithm>` 庫中的 `std::sort()`。因為 `Student` 是自訂結構，我傳入了一個自訂的 Lambda 運算式作為比較依據：

```
void sortStudents() { if (students.empty()) return; // 使用
C++17 Lambda 進行自訂排序(由高到低)
sort(students.begin(), students.end(), [](const Student& a,
const Student& b) { return a.score > b.score; }); cout <<
"[SUCCESS] Sorted by score (High to Low)! Use Option
2 to view.\n"; }
```

5.2 Template 挑戰題的整合與呼叫 (功能 5)

作業規定必須設計泛型模板函式 `getMax` 與 `getMin`，並實際整合進統計功能中。我將其應用在遍歷 `vector` 計算全班最高分與最低分的邏輯中：

C++

```
void showStatistics() {  
  
    if (students.empty()) return;  
  
    int sum = 0;  
  
    int maxScore = students[0].score;  
  
    int minScore = students[0].score;  
  
    for (const auto& s : students) {  
  
        sum += s.score;  
  
        // 實際呼叫 Template 挑戰題函式  
  
        maxScore = getMax<int>(maxScore, s.score);  
  
        minScore = getMin<int>(minScore, s.score);  
  
        // ...其餘統計邏輯...  
  
    }  
  
}
```

6. 執行畫面或輸出結果

```
● PS C:\Student> g++ -std=c++17 main.cpp -o main
○ PS C:\Student> .\main.exe
```

```
===== Student Management System =====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
```

```
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
Enter option: 1
Enter Student ID: S11259032
Enter Name: Cubee
Enter Score: 99
[SUCCESS] Student data added successfully!
```



```
4. Search by id
5. Show statistics
0. Exit
```

```
=====
```

```
Enter option: 2
```

```
-----
ID           Name           Score
-----
S11259032    Cubee           99
-----
```

```
=====
```

```
Enter option: 4
```

```
Enter Student ID to search: S11259032
```

```
-----
[RESULT] ID: S11259032 | Name: Cubee | Score: 99
-----
```

```
===== Student Management System =====
```

```
-----  
Enter option: 5
```

```
-----  
Class Average : 99.00
```

```
Highest Score : 99
```

```
Lowest Score  : 99
```

```
Passed Count  : 1
```

```
Failed Count  : 0  
-----
```

7. GitHub Repository [連結與檔案結構說明](#)

your-repository/

|—— main.cpp # 包含核心業務邏輯、
Student 結構與 Template 挑戰題的主程式

|—— README.md # 包含專案簡介、g++
編育與執行指令的 Markdown 說明文件

|—— report.pdf # 本份自主學習結案報告
(不含封面共 8 頁)

8.遇到的問題及解決問題的方式

在自主學習與小專題開發的過程中，我遇到了以下兩個具體的技術瓶頸，並透過查閱官方文件與反覆偵錯順利解決：

(1) Template 函式與自訂變數型別的嚴格匹配問題

- **問題描述：**在實作功能 5 統計成績時，我宣告了 `int maxScore = students[0].score;`。

但在主程式計算平均值時，我一開始將某些統計暫存變數混用了 `double`。當我試圖呼叫挑戰題的模板函式 `maxScore = getMax(maxScore, s.score);` 時，編譯器突然噴出一大堆泛型匹配錯誤（模版推導失敗）。

- **原因分析：**C++ 的 Template 在進行型別推導時極度嚴格，不允許隱式轉型（Implicit Conversion）。如果傳入的兩個參數型別不完全一致，編譯器就會拒絕通過。
- **解決方式：**我將所有參與最高分、最低分比較的變數統一限制為 `int` 型別，並在呼叫時明確指定樣板參數 `getMax<int>(maxScore, s.score)`。如此一來，編譯器便能精準進行實例化，順利通過 `g++ -std=c++17` 的編譯。

(2) Windows 終端機 (CMD/PowerShell) 編碼衝突導致中文亂碼

- **問題描述：**最初為了讓介面友善，程式碼中的選單與提示字串（如「學生成績管理系統」、

「請輸入選項」) 皆採用繁體中文撰寫。但在 Windows 環境下編譯執行時，終端機畫面上卻噴出一整串如 σjητöfμêÉ 的亂碼，嚴重影響操作與畫面截圖的完整度。

- **原因分析：**這是因為我的原始碼檔案是採用現代通用的 **UTF-8** 編碼儲存，而 Windows 台灣地區的終端機預設編碼卻是傳統的 **Big5**。兩者在解析中文字元時產生衝突，進而導致解碼失敗。
- **解決方式：**雖然可以透過指令 `chcp 65001` 強制切換終端機編碼，但考量到跨平台評分的相容性，且為了讓專案更具國際化與專業工程師的思維，我決定將系統的所有 CLI 提示文字、錯誤訊息與表格排版全面重構成「全英文介面」(如 Enter Student ID:, [SUCCESS])。這不僅徹底根治了環境亂碼問題，也讓最終產出的執行截圖畫面乾淨漂亮。

9.

(1) 學習心得與反思

這兩週的自主學習真的很有成就感！以前上課學 Template 都覺得只是應付考試的抽象語法，直到這週親手用 `std::vector` 寫出這個管理系統，才發現原來不用重複造輪子這麼痛快。最酷的是，我私底下有在當國中生的物化家教，這次做完，我剛好能把這個系統改成專屬於他們的成績追蹤小工具。看到程式碼能實際解決生活中的問題，真的超有工程師的感動！

(2) 未來可延伸方向

下學期就要正式上「資料結構」和「演算法」這些重頭戲了，這份作業算是提早幫我推開了現代 C++ 的大門。之後除了想幫這個系統加上檔案 I/O（讓家教資料可以存成 `.csv` 檔）之外，在課堂上我也不想只會「呼叫」容器，更想去研究 `vector` 內部是怎麼動態配記憶體的，還有 `map` 底層的紅黑樹

到底是怎麼平衡的，提早幫下學期的硬課打好信心！